

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME:Database Management Systems

COURSE CODE:CSA05

NAME:P.Jessica Hephzbah

REG NO: 192511425

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High) Assessment

Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding ; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with	Mostly correct steps with minor	Disorganized steps; multiple errors	No clear process
------------------	-----	--------------------------------	---------------------------------	-------------------------------------	------------------

I P.Jessica Hephzbah(192511425) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Jessica

RUBRICS FOR CALCULATION:

		correct approach throughout	mistakes		
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1)

Introduction

Relational algebra is a procedural query language used to retrieve and manipulate data stored in relational databases.

Interpretation

The Employee and Department relations are used to demonstrate selection, projection, union, difference, Cartesian product, and join operations.

Problem Statement

Given Employee and Department relations, apply basic relational algebra operations to retrieve required records.

Employee Relation

EmpID	EmpName	Salary	DeptID
101	John	50000	1
102	Smith	60000	2
103	David	45000	1

Department Relation

DeptID	DeptName
1	CSE
2	ECE
3	MECH

1. Selection (σ)

Find employees having salary greater than 50000

Relational Algebra:

$\sigma_{\text{Salary} > 50000}(\text{Employee})$

SQL:

```
SELECT *  
FROM Employee  
WHERE Salary > 50000;
```

Output:

EmpID	EmpName	Salary
102	Smith	60000

2. Projection (π)

Display employee names

Relational Algebra:

$\pi_{\text{EmpName}}(\text{Employee})$

SQL:

```
SELECT EmpName
FROM Employee;
```

Output:

EmpName
John
Smith
David

3. Union ($\cup$)

Relational Algebra:

$\text{Employee} \cup \text{Employee2}$

SQL:

```
SELECT * FROM Employee
UNION
SELECT * FROM Employee2;
```

4. Difference ($-$)

Find records present in Employee but not in Employee2

Relational Algebra:

$\text{Employee} - \text{Employee2}$

SQL:

```
SELECT *  
FROM Employee  
WHERE EmpID NOT IN  
(SELECT EmpID FROM Employee2);
```

5. Cartesian Product ($\times$)

Relational Algebra:

Employee $\times$ Department Employee $\bowtie$ Department Employee $\times$ Department

SQL:

```
SELECT *  
FROM Employee  
CROSS JOIN Department;
```

6. Join ($\bowtie$)

Combine Employee and Department using DeptID

Relational Algebra:

Employee $\bowtie$ Department Employee $\bowtie$ Department Employee $\bowtie$ Department

SQL:

```
SELECT EmpName, DeptName  
FROM Employee  
INNER JOIN Department  
ON Employee.DeptID = Department.DeptID;
```

2)

Introduction

DDL (Data Definition Language) is used to define and create database structures such as tables with constraints.

Interpretation

The Student and Department tables are created with primary keys and foreign key relationships to maintain data integrity.

Problem Statement

Create **STUDENT** and **DEPARTMENT** tables with suitable constraints like PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE.

SQL Code

Create Department Table

```
CREATE TABLE DEPARTMENT
(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

Create Student Table

```
CREATE TABLE STUDENT
(
    SID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    DOB DATE NOT NULL,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT (DeptID)
);
```

Output

Department Table Created

DeptID	DeptName
101	CSE
102	ECE
103	MECH

Student Table Created

SID	Name	DOB	DeptID
1	John	2005-05-10	101
2	Smith	2005-08-15	102
3	David	2004-12-20	101

Conclusion

The STUDENT and DEPARTMENT tables are successfully created using SQL DDL commands with appropriate constraints to ensure data consistency and integrity

3)

Introduction

Aggregate functions in SQL are used to perform calculations on a group of records and summarize data.

Interpretation

The Sales table is analyzed using GROUP BY and HAVING clauses to calculate total sales, average sales, and other statistical values.

Problem Statement

Given a Sales dataset, write SQL queries using **COUNT**, **SUM**, **AVG**, **MAX**, **MIN**, **GROUP BY**, and **HAVING**.

Sales Table

SaleID	Product	Category	Amount
1	Laptop	Electronics	50000
2	Mobile	Electronics	30000
3	Chair	Furniture	10000
4	Table	Furniture	20000
5	TV	Electronics	40000

1. COUNT() - Count total sales records

```
SELECT COUNT(*) AS Total_Sales
FROM Sales;
```

Output

Total_Sales
5

2. SUM() - Find total sales amount

```
SELECT SUM(Amount) AS Total_Amount
FROM Sales;
```

Output

Total_Amount
150000

3. AVG() - Find average sales amount

```
SELECT AVG(Amount) AS Average_Amount
FROM Sales;
```

Output

Average_Amount
30000

4. MAX() and MIN() - Find highest and lowest sales

```
SELECT MAX(Amount) AS Maximum_Sale,
       MIN(Amount) AS Minimum_Sale
FROM Sales;
```

Output

Maximum_Sale	Minimum_Sale
50000	10000

5. GROUP BY - Calculate category-wise sales

```
SELECT Category, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Category;
```

Output

Category	Total_Sales
Electronics	120000
Furniture	30000

6. HAVING - Display categories having sales above 50000

```
SELECT Category, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Category
HAVING SUM(Amount) > 50000;
```

Output

Category	Total_Sales
Electronics	120000

Conclusion

GROUP BY, HAVING, and aggregate functions help to analyze sales data by performing calculations and retrieving meaningful summarized information from databases

4)

Introduction

A correlated sub-query is a query that executes once for each row of the outer query and uses values from the outer query.

Interpretation

The employee salary is compared with the average salary of their respective department to find higher-paid employees.

Problem Statement

Find employees whose salary is greater than the average salary of their own department.

Employee Table

EmpID	EmpName	Salary	DeptID
101	John	50000	1
102	Smith	60000	2
103	David	45000	1
104	Mary	70000	3
105	Alex	55000	2

SQL Query

```
SELECT E.EmpID, E.EmpName, E.Salary, E.DeptID
FROM Employee E
WHERE E.Salary >
(
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E.DeptID
);
```

Output

EmpID	EmpName	Salary	DeptID
101	John	50000	1
102	Smith	60000	2
104	Mary	70000	3

Explanation

- The inner query calculates the average salary of each employee's department.
- The outer query compares each employee salary with that department average.
- Employees earning more than their department average are displayed.

Conclusion

The correlated sub-query successfully retrieves employees whose salary is higher than the average salary of their respective departments.

5)

Introduction

JOIN operations in SQL are used to combine data from two or more tables based on a related column.

Interpretation

Employee and Project tables are joined using the common attribute **EmpID** to retrieve related information.

Problem Statement

Perform different types of JOIN operations between Employee and Project tables.

Employee Table

EmpID	EmpName	DeptID
101	John	1
102	Smith	2
103	David	1
104	Mary	3

Project Table

ProjectID	ProjectName	EmpID
201	AI System	101
202	Website	102

203	Database	105
-----	----------	-----

1. INNER JOIN

Displays only matching records from both tables.

SQL Query

```
SELECT Employee.EmpID, EmpName, ProjectName
FROM Employee
INNER JOIN Project
ON Employee.EmpID = Project.EmpID;
```

Output

EmpID	EmpName	ProjectName
101	John	AI System
102	Smith	Website

2. LEFT OUTER JOIN

Displays all Employee records and matching Project details.

SQL Query

```
SELECT Employee.EmpID, EmpName, ProjectName
FROM Employee
LEFT JOIN Project
ON Employee.EmpID = Project.EmpID;
```

Output

EmpID	EmpName	ProjectName
101	John	AI System
102	Smith	Website
103	David	NULL
104	Mary	NULL

3. RIGHT OUTER JOIN

Displays all Project records and matching Employee details.

SQL Query

```
SELECT Employee.EmpID, EmpName, ProjectName
FROM Employee
RIGHT JOIN Project
ON Employee.EmpID = Project.EmpID;
```

Output

EmpID	EmpName	ProjectName
101	John	AI System
102	Smith	Website
NULL	NULL	Database

4. FULL OUTER JOIN

Displays all records from both tables.

SQL Query

```
SELECT Employee.EmpID, EmpName, ProjectName
FROM Employee
FULL OUTER JOIN Project
ON Employee.EmpID = Project.EmpID;
```

Output

EmpID	EmpName	ProjectName
101	John	AI System
102	Smith	Website
103	David	NULL
104	Mary	NULL
NULL	NULL	Database

5. CROSS JOIN

Displays Cartesian product of both tables.

SQL Query

```
SELECT EmpName, ProjectName
FROM Employee
CROSS JOIN Project;
```

Output

EmpName	ProjectName
John	AI System
John	Website
John	Database
Smith	AI System
Smith	Website

Smith	Database
-------	----------

Conclusion

Different JOIN operations help combine and retrieve related data from multiple tables based on specific requirements.

6)

Introduction

A view in SQL is a virtual table created from a query to simplify data retrieval and provide customized information.

Interpretation

The view **DeptSalaryView** combines department details with employee salary information to display department-wise employee count and average salary.

Problem Statement

Create a view that shows:

- Department Name
- Total number of employees
- Average salary of employees

Employee Table

EmpID	EmpName	Salary	DeptID
101	John	50000	1
102	Smith	60000	2
103	David	45000	1
104	Mary	70000	3

Department Table

DeptID	DeptName
1	CSE
2	ECE
3	MECH

SQL Code to Create View

```
CREATE VIEW DeptSalaryView AS
SELECT
    D.DeptName,
    COUNT(E.EmpID) AS Total_Employees,
    AVG(E.Salary) AS Average_Salary
FROM Department D
JOIN Employee E
ON D.DeptID = E.DeptID
GROUP BY D.DeptName;
```

Query on View

```
SELECT *
FROM DeptSalaryView;
```

Output

DeptName	Total_Employees	Average_Salary
CSE	2	47500
ECE	1	60000
MECH	1	70000

Conclusion

The view **DeptSalaryView** is successfully created and used to display department-wise employee count and average salary, improving data accessibility and query simplicity

7)

Introduction

Relational algebra provides operations to retrieve and manipulate data from relational database tables.

Interpretation

The division ($\div$) operation in relational algebra is used to find students who are associated with **all records** in another relation.

Problem Statement

Find the names of students who are enrolled in all courses offered by the institution.

Relations

Student

SID	Student_Name
101	John
102	Smith
103	David

Course

CID	Course_Name
C1	DBMS
C2	OS
C3	AI

Enrollment

SID	CID
101	C1
101	C2
101	C3
102	C1
102	C2
103	C1

Relational Algebra Expression

First find students enrolled in all courses:

$\text{Enrollment} \div \text{Course}$

This gives the **SID** of students who have enrolled in every course.

Then retrieve student names:

$\pi_{\text{Student_Name}}(\text{Student} \bowtie (\text{Enrollment} \div \text{Course}))$

SQL Equivalent

```
SELECT S.Student_Name
FROM Student S
WHERE NOT EXISTS
(
    SELECT C.CID
    FROM Course C
    WHERE NOT EXISTS
    (
        SELECT E.CID
        FROM Enrollment E
        WHERE E.SID = S.SID
        AND E.CID = C.CID
    )
);
```

Output

Student_Name
John

Conclusion

The relational algebra division operation is used to identify students who are enrolled in every available course, and the equivalent SQL query retrieves the required student names.

8)

Introduction

DML (Data Manipulation Language) is used to insert, modify, and delete records in database tables.

Interpretation

INSERT, UPDATE, and DELETE commands are applied on a table to manage the stored data.

Problem Statement

Perform DML operations on an Employee table using INSERT, UPDATE, and DELETE commands.

Employee Table

EmpID	EmpName	Salary	DeptID
101	John	50000	1
102	Smith	60000	2
103	David	45000	1

1. INSERT Command

Used to add new records into a table.

SQL Query

```
INSERT INTO Employee  
(EmpID, EmpName, Salary, DeptID)  
VALUES  
(104, 'Mary', 70000, 3);
```

Output

EmpID	EmpName	Salary	DeptID
104	Mary	70000	3

2. UPDATE Command

Used to modify existing records.

SQL Query

```
UPDATE Employee  
SET Salary = 75000  
WHERE EmpID = 104;
```

Output

EmpID	EmpName	Salary	DeptID
104	Mary	75000	3

3. DELETE Command

Used to remove records from a table.

SQL Query

```
DELETE FROM Employee  
WHERE EmpID = 104;
```

Output

EmpID	EmpName	Salary	DeptID
101	John	50000	1
102	Smith	60000	2
103	David	45000	1

Conclusion

DML commands such as INSERT, UPDATE, and DELETE are used to manage and maintain data stored in database tables efficiently